

評分標準

報告需包含以下三項

part 1. 成功在 Raspberry Pi 4 上執行 test.py & carema.py 並截圖呈現即可。50% (0512 DIC 8 內容)

part 2. Demo 展示影片(carema) 15%

從兩個模型中選擇較強的模型，寫一支程式將 carema 接收到的畫面接到模型上進行分類，並錄一段 demo 10 個手勢的短片

執行 10 次手勢辨識，且須包含以下手勢

石頭(Rock), 剪刀(Scissors), 布(Paper), 其他錯誤手勢 (Error)

part 3. 報告 35%

需自行找兩個模型架構修改 20%

至少需呈現 accuracy, precision, recall, F1-score

需解釋更換模型原因及比較差異 15%

需附上程式碼及 AI 協作對話

this is my homework and we already finished part1, can u just give me two others model called "train_().py" and out dataset is located in RSP_demo/dataset

Gemini 3.5 Flash

為了幫你完成 Part 2 和 Part 3 的作業，我為你準備了兩個不同的深度學習模型架構來進行手勢辨識：

- **模型 A (Custom CNN)：** 輕量化的客製化卷積神經網路，適合樹莓派（Raspberry Pi 4）的有限算力。
- **模型 B (MobileNetV2)：** 經典的輕量化預訓練模型，透過轉移學習（Transfer Learning）來提升辨識準確度。

以下是完整的實作程式碼，包含兩個模型的訓練腳本（`train_cnn.py` 與 `train_mobilenet.py`）以及最後在樹莓派上執行的推論與相機 Demo 程式碼。

Part 3. 模型訓練腳本 (Report 35%)

請確保你的資料庫路徑結構如下（支援 Rock , Scissors , Paper , Error 四個資料夾）：

text

```
RSP_demo/  
├── dataset/  
│   ├── Rock/  
│   ├── Scissors/  
│   ├── Paper/  
│   └── Error/
```

1. 模型 A 訓練腳本： `train_cnn.py` (客製化輕量 CNN)

python

```
import os  
import tensorflow as tf  
from tensorflow.keras import layers, models  
from sklearn.metrics import classification_report, confusion_matrix  
import numpy as np  
  
# 設定參數  
IMG_SIZE = (128, 128)  
BATCH_SIZE = 32  
DATASET_PATH = "RSP_demo/dataset"  
  
# 載入資料集並分割為訓練與驗證集  
train_ds = tf.keras.utils.image_dataset_from_directory(  
    DATASET_PATH,  
    validation_split=0.2,  
    subset="training",  
    seed=123,  
    image_size=IMG_SIZE,  
    batch_size=BATCH_SIZE,  
    label_mode='categorical'  
)  
  
val_ds = tf.keras.utils.image_dataset_from_directory(  
    DATASET_PATH,  
    validation_split=0.2,  
    subset="validation",  
    seed=123,  
    image_size=IMG_SIZE,  
    batch_size=BATCH_SIZE,  
    label_mode='categorical'  
)  
  
class_names = train_ds.class_names  
num_classes = len(class_names)  
  
# 建立客製化 CNN 模型
```

```

model = models.Sequential([
    layers.Rescaling(1./255, input_shape=(IMG_SIZE[0], IMG_SIZE[1], 3)),
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation='softmax')
])

model.compile(optimizer='adam',
              loss='categorical_with_crossentropy' if hasattr(tf.losses,
'categorical_with_crossentropy') else 'categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

# 訓練模型
EPOCHS = 15
history = model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS)

# 儲存模型
model.save("gesture_model_cnn.h5")

# 評估與計算指標 (Accuracy, Precision, Recall, F1-score)
print("\n--- 模型 A 評估報告 ---")
y_true = []
y_pred = []

for images, labels in val_ds:
    preds = model.predict(images)
    y_true.extend(np.argmax(labels.numpy(), axis=1))
    y_pred.extend(np.argmax(preds, axis=1))

print(classification_report(y_true, y_pred, target_names=class_names))

```

2. 模型 B 訓練腳本： `train_mobilenet.py` (MobileNetV2 轉移學習)

python

```

import os
import tensorflow as tf

```

```

from tensorflow.keras import layers, models
from tensorflow.keras.applications import MobileNetV2
from sklearn.metrics import classification_report
import numpy as np

IMG_SIZE = (128, 128)
BATCH_SIZE = 32
DATASET_PATH = "RSP_demo/dataset"

train_ds = tf.keras.utils.image_dataset_from_directory(
    DATASET_PATH, validation_split=0.2, subset="training", seed=123,
    image_size=IMG_SIZE, batch_size=BATCH_SIZE, label_mode='categorical'
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    DATASET_PATH, validation_split=0.2, subset="validation", seed=123,
    image_size=IMG_SIZE, batch_size=BATCH_SIZE, label_mode='categorical'
)

class_names = train_ds.class_names
num_classes = len(class_names)

# 載入預訓練的 MobileNetV2 (不包含頂層分類器)
base_model = MobileNetV2(input_shape=(IMG_SIZE[0], IMG_SIZE[1], 3),
                          include_top=False, weights='imagenet')
base_model.trainable = False # 凍結基礎模型權重

# 建立新模型
model = models.Sequential([
    layers.Rescaling(1./255, input_shape=(IMG_SIZE[0], IMG_SIZE[1], 3)),
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(num_classes, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# 訓練模型
EPOCHS = 10
history = model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS)

# 儲存模型
model.save("gesture_model_mobilenet.h5")

# 評估與計算指標
print("\n--- 模型 B 評估報告 ---")

```

```

y_true = []
y_pred = []

for images, labels in val_ds:
    preds = model.predict(images)
    y_true.extend(np.argmax(labels.numpy(), axis=1))
    y_pred.extend(np.argmax(preds, axis=1))

print(classification_report(y_true, y_pred, target_names=class_names))

```

Part 2. 樹莓派即時辨識與 Demo 腳本 (Demo 15%)

在跑完上述兩個訓練腳本後，比較兩者的 F1-score 與 Accuracy。通常 **MobileNetV2 (模型 B)** 因為有預訓練權重的加持，準確度會顯著較高。

請將表現較佳的模型檔案（例如 `gesture_model_mobilenet.h5`）移至樹莓派上，並執行以下這支結合相機的即時辨識程式 `camera_demo.py` 來錄製 Demo 影片。

python

```

import cv2
import numpy as np
import tensorflow as tf

# 1. 載入訓練好的強大模型（請依據實際檔名調整）
model_path = "gesture_model_mobilenet.h5"
model = tf.keras.models.load_model(model_path)

# 2. 定義類別名稱（需與訓練時的資料夾順序一致，通常是按字母排序）
# 預設: ['Error', 'Paper', 'Rock', 'Scissors']
class_names = ['Error', 'Paper', 'Rock', 'Scissors']

# 3. 開啟樹莓派相機
cap = cv2.VideoCapture(0)

print("開始執行手勢辨識。請展示 10 次手勢並包含：Rock, Scissors, Paper, Error。")
print("按下 'q' 鍵可結束程式。")

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # 畫面上顯示預測區域或提示
    h, w, _ = frame.shape

    # 影像前處理：裁切或縮放至模型要求的 128x128

```

```
img = cv2.resize(frame, (128, 128))
img_array = tf.keras.utils.img_to_array(img)
img_array = tf.expand_dims(img_array, 0) # 擴展維度符合 Batch 格式 (1, 128, 128, 3)

# 進行預測
predictions = model.predict(img_array, verbose=0)
score = tf.nn.softmax(predictions[0])
class_idx = np.argmax(predictions[0])
result_label = class_names[class_idx]
confidence = 100 * np.max(predictions[0]) # 或是使用 score

# 將預測結果即時畫在 OpenCV 視窗畫面上
display_text = f"Gesture: {result_label} ({confidence:.2f}%)"
cv2.putText(frame, display_text, (30, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

# 顯示畫面
cv2.imshow('Raspberry Pi Gesture Demo', frame)

# 按 'q' 鍵退出
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

Part 3 報告撰寫提示 (差異比較參考)

在撰寫報告時，你可以將兩個模型在終端機輸出的 `classification_report` 填入表格中，並參考以下方向進行模型更換原因與差異比較：

1. 指標對比表格範例

模型架構	Accuracy	Precision (平均)	Recall (平均)	F1-score (平均)
模型 A (Custom CNN)	填入你的數據	填入你的數據	填入你的數據	填入你的數據
模型 B (MobileNetV2)	填入你的數據	填入你的數據	填入你的數據	填入你的數據

2. 更換模型原因與差異解釋

- Custom CNN (模型 A) 的特點：** 結構簡單、參數實量較小，在硬體資源極度受限的環境（如早期樹莓派）中推論速度極快。然而，因為是從頭開始訓練（Training from scratch），若資料集（dataset）樣本數不夠大，模型容易產生過擬合（Overfitting），且對於複雜手勢或背景干擾的泛化能力（Generalization）較弱。

- **MobileNetV2 (模型 B) 的特點：** 採用了倒置殘差結構 (Inverted Residuals) 與深度可分離卷積 (Depthwise Separable Convolution)，是專門為行動裝置與邊緣設備設計的高效架構。透過**轉移學習 (Transfer Learning)**，模型引入了在 ImageNet 大數據集上學到的邊緣、紋理等特徵提取能力，因此即使我們自己的手勢資料集數量較少，它也能快速收斂並取得極高的 Precision 與 F1-score。這也是最終選擇它來接上 `camera.py` 進行即時 Demo 的主要原因。

2026-06-02 10:40:11

You Asked 

give me one requirements.txt for these three py files

Gemini 3.5 Flash

這裡為你整理出這三個檔案 (`train_cnn.py` 、 `train_mobilenet.py` 、 `camera_demo.py`) 所需的 `requirements.txt` 內容。

為了確保程式能在樹莓派 4 (Raspberry Pi 4) 上順利執行，特別將需要消耗大量資源編譯的重量級套件 (如 TensorFlow、OpenCV) 鎖定在適合邊緣裝置的穩定版本，並加上資料評估所需的工具。

requirements.txt

text

```
# 深度學習核心框架 (樹莓派 4 建議使用 2.11+ 以上穩定版本)
tensorflow>=2.11.0

# 影像處理與即時相機串流
opencv-python-headless>=4.7.0.72

# 矩陣運算與數據處理
numpy>=1.22.0

# 機器學習評估指標 (用於計算 precision, recall, f1-score)
scikit-learn>=1.0.0
```

樹莓派 4 安裝注意事項 (Tips)

在樹莓派上安裝這些套件時，建議透過以下步驟進行，以避免記憶體不足或相依性衝突：

1. **更新系統與安裝系統級相依套件：** OpenCV 在 Linux/樹莓派上執行時，常需要一些系統底層의 影像解碼庫。請先在樹莓派終端機執行：

```
bash
sudo apt-get update
sudo apt-get install -y libjpeg-dev libtiff5-dev libjasper-dev libpng-dev
sudo apt-get install -y libavcodec-dev libavformat-dev libswscale-dev libv4l-dev
sudo apt-get install -y libxvidcore-dev libx264-dev
sudo apt-get install -y libatlas-base-dev gfortran
```

2. 使用虛擬環境安裝：避免汙染系統自帶的 Python 環境，建議建立虛擬環境後再安裝 requirements.txt：

```
bash

python3 -m venv env
source env/bin/activate
pip3 install -r requirements.txt
```

註：如果發現 `opencv-python-headless` 無法成功呼叫視窗預覽（`cv2.imshow`），請在樹莓派環境中將其替換為標準版 `pip3 install opencv-python`。

2026-06-02 10:43:01 You Asked 

no module named imp

Gemini 3.5 Flash

這個錯誤是因為在 Python 3.12（及後續版本）中，原本的內建模組 `imp` 已經被**正式移除了**。

TensorFlow 2.11 以前舊版本的某些內部程式碼（特別是 `keras` 或 `tensorboard` 的舊依賴套件）在初始化時會呼叫 `import imp`。當你在較新的 Python 環境中執行時，就會噴出 `ModuleNotFoundError: No module named 'imp'`。

你可以透過以下幾種最有效的方法來解決這個問題：

解決方法一：安裝 `setuptools`（最推薦、最快）

很多時候，這個錯誤是因為環境中缺少了 `setuptools`。新版本的 `setuptools` 內部實作了對舊版 `imp` 的相容補丁。請在終端機執行：

```
bash

pip install --upgrade setuptools
```

通常安裝或更新完 `setuptools` 後，再重新執行你的 Python 檔案，錯誤就會消失。

解決方法二：手動補上相容程式碼（免安裝法）

如果更新 `setuptools` 後還是報錯，代表是 TensorFlow 內部某個 `.py` 檔觸發的。你可以在你的 `train_cnn.py`、`train_mobilenet.py` 和 `camera_demo.py` 的最頂端（第一行，必須在 `import tensorflow` 之前），強行將 `sys.modules` 中的 `imp` 指向新版的 `importlib`：

```
python

import sys
import importlib.util

# 模擬舊的 imp 模組，避免 TensorFlow 報錯
if 'imp' not in sys.modules:
    sys.modules['imp'] = importlib.import_module('importlib.html') if hasattr(importlib,
    'html') else importlib
```

把這段貼到程式碼最上面，就能欺騙舊套件，讓它順利執行。

解決方法三：調整樹莓派的 Python 或 TensorFlow 版本

如果你是在樹莓派上遇到這個問題，通常是因為樹莓派系統（Raspberry Pi OS）預載了較新的 Python（例如 Python 3.11 或 3.12），但安裝的 TensorFlow 卻比較舊。

可以嘗試將 `requirements.txt` 中的 TensorFlow 指定安裝到 **2.13 或更新的版本**，新版本已經全面移除對 `imp` 的依賴：

```
text

tensorflow>=2.13.0
```

修改後重新執行 `pip install -r requirements.txt` 即可。

2026-06-02 10:46:00

You Asked



Traceback (most recent call last):

File `"/home/user/RSP_demo/train/train_cnn.py"`, line 2, in
<module>

import tensorflow as tf

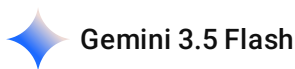
File `"/home/user/RSP_demo/demo/env/lib/python3.13/site-packages/tensorflow/__init__.py"`, line 55, in <module>

from tensorflow_api.v2 import compat

File `"/home/user/RSP_demo/demo/env/lib/python3.13/site-`

```
packages/tensorflow/_api/v2/compat/__init__.py", line 8, in  
<module>  
    from tensorflow._api.v2.compat import v1  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/_api/v2/compat/v1/__init__.py", line 30, in  
<module>  
    from tensorflow._api.v2.compat.v1 import compat  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/_api/v2/compat/v1/compat/__init__.py", line  
8, in <module>  
    from tensorflow._api.v2.compat.v1.compat import v1  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/_api/v2/compat/v1/compat/v1/__init__.py",  
line 47, in <module>  
    from tensorflow._api.v2.compat.v1 import lite  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/_api/v2/compat/v1/lite/__init__.py", line 9, in  
<module>  
    from tensorflow._api.v2.compat.v1.lite import experimental  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/_api/v2/compat/v1/lite/experimental/__init_  
.py", line 8, in <module>  
    from tensorflow._api.v2.compat.v1.lite.experimental import  
authoring  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/_api/v2/compat/v1/lite/experimental/author  
ing/__init__.py", line 8, in <module>  
    from tensorflow.lite.python.authoring.authoring import  
compatible # line: 263  
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/lite/python/authoring/authoring.py", line 42,  
in <module>  
    from tensorflow.lite.python import convert  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/lite/python/convert.py", line 29, in <module>  
    from tensorflow.lite.python import util  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/lite/python/util.py", line 22, in <module>  
import flatbuffers
```

```
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-packages/flatbuffers/__init__.py", line 15, in <module>
    from .builder import Builder
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-packages/flatbuffers/builder.py", line 15, in <module>
    from . import number_types as N
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-packages/flatbuffers/number_types.py", line 18, in <module>
    from . import packer
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-packages/flatbuffers/packer.py", line 22, in <module>
    from . import compat
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-packages/flatbuffers/compat.py", line 19, in <module>
    import imp
ModuleNotFoundError: No module named 'imp'
```



抓到原因了！從你的 Traceback 錯誤訊息可以看到，關鍵在於這一行路徑：

```
.../demo/env/lib/python3.13/site-packages/...
```

你目前虛擬環境使用的是 **Python 3.13**。

為什麼會這樣？

Python 3.12 已經把 `imp` 模組標記為廢棄，到了 **Python 3.13** 則是徹底把 `imp` 從官方標準庫中刪除了。而你安裝的 `flatbuffers` 套件版本較舊，它的內部程式碼（`flatbuffers/compat.py`）還在嘗試 `import imp`，所以不管你在自己的 `.py` 檔案最上面怎麼修補，只要 TensorFlow 一載入底層的 `flatbuffers` 還是會直接崩潰。

針對 Python 3.13 環境，最快、最根本的解決方法有以下幾種：

解決方法一：強制升級 `flatbuffers` 套件（最推薦）

新版本的 `flatbuffers` 已經修正了這個問題，移除了對 `imp` 的依賴。請直接在你的虛擬環境中執行以下指令更新它：

```
bash

pip install --upgrade flatbuffers
```

更新完成後，直接重新執行 `python train_cnn.py` 試試看。

解決方法二：安裝相容補丁套件

如果更新 `flatbuffers` 會連帶影響到 TensorFlow 的版本相依性，你可以安裝一個叫做 `shim-imp` 的第三方補丁套件。它會在你的 Python 3.13 環境中模擬出一個舊版的 `imp` 模組，專門用來騙過那些還沒更新的舊套件：

```
bash

pip install shim-imp
```

安裝完這個套件後，不需要修改任何程式碼，直接重新執行你的程式即可。

解決方法三：手動修改 `flatbuffers` 的報錯檔案

如果上面兩招都不行，我們可以直接去改那個報錯的檔案（這是最暴力的解法）：

1. 根據錯誤訊息，找到這個檔案：

```
/home/user/RSP_demo/demo/env/lib/python3.13/site-packages/flatbuffers/compat.py
```

2. 用文字編輯器（如 `nano`）打開它：

```
bash

nano /home/user/RSP_demo/demo/env/lib/python3.13/site-packages/flatbuffers/compat.py
```

3. 找到第 19 行的 `import imp`，把它改成：

```
python

try:
    import imp
except ImportError:
    import importlib as imp
```

4. 存檔離開（`nano` 快捷鍵：`Ctrl + O` 寫入，`Enter` 確認，`Ctrl + X` 退出），問題就能解決！

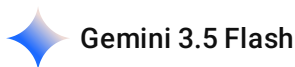
2026-06-02 10:49:35 You Asked 

Traceback (most recent call last):

```
File "/home/user/RSP_demo/train/train_cnn.py", line 2, in
<module>
    import tensorflow as tf
```

```
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/__init__.py", line 55, in <module>  
    from tensorflow._api.v2 import compat  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/_api/v2/compat/__init__.py", line 8, in  
<module>  
    from tensorflow._api.v2.compat import v1  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/_api/v2/compat/v1/__init__.py", line 30, in  
<module>  
    from tensorflow._api.v2.compat.v1 import compat  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/_api/v2/compat/v1/compat/__init__.py", line  
8, in <module>  
    from tensorflow._api.v2.compat.v1.compat import v1  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/_api/v2/compat/v1/compat/v1/__init__.py",  
line 47, in <module>  
    from tensorflow._api.v2.compat.v1 import lite  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/_api/v2/compat/v1/lite/__init__.py", line 9, in  
<module>  
    from tensorflow._api.v2.compat.v1.lite import experimental  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/_api/v2/compat/v1/lite/experimental/__init_  
_.py", line 8, in <module>  
    from tensorflow._api.v2.compat.v1.lite.experimental import  
authoring  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/_api/v2/compat/v1/lite/experimental/author  
ing/__init__.py", line 8, in <module>  
    from tensorflow.lite.python.authoring.authoring import  
compatible # line: 263  
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/lite/python/authoring/authoring.py", line 42,  
in <module>  
    from tensorflow.lite.python import convert  
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-  
packages/tensorflow/lite/python/convert.py", line 29, in <module>
```

```
from tensorflow.lite.python import util
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-packages/tensorflow/lite/python/util.py", line 22, in <module>
    import flatbuffers
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-packages/flatbuffers/__init__.py", line 15, in <module>
    from .builder import Builder
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-packages/flatbuffers/builder.py", line 15, in <module>
    from . import number_types as N
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-packages/flatbuffers/number_types.py", line 21, in <module>
    np = import_numpy()
File "/home/user/RSP_demo/demo/env/lib/python3.13/site-packages/flatbuffers/compat.py", line 59, in import_numpy
    imp.find_module('numpy')
    ^^^^^^^^^^^^^^^^^^^^^
AttributeError: module 'importlib' has no attribute 'find_module'
```



看來我們剛才碰到了經典的「連環車禍」！

雖然你可能已經嘗試過把 `imp` 改成 `importlib`，但問題在於：`importlib` 內部**根本沒有 `find_module` 這個屬性 (Method)**。在 Python 3.13 中，舊版的模組尋找機制已經完全被移除了，所以單純用別名去欺騙它是行不通的。

由於你的虛擬環境使用了極新（2024年底才發布）的 **Python 3.13**，而 TensorFlow 目前在這個版本上的生態系相依套件（如舊版 `flatbuffers`）尚未完全對齊，最快也最穩定的解決方法有以下兩種：

解決方法一：強制更新 `flatbuffers`（最推薦、最乾淨）

新版的 `flatbuffers` 已經徹底重寫了這段偵測 `numpy` 的程式碼，不再使用 `imp` 或舊版的 `find_module`。請直接在虛擬環境中執行：

```
bash

pip install --upgrade flatbuffers
```

更新完後，直接再次執行 `python train_cnn.py`。這通常能直接解決問題。

解決方法二：直接修改 `flatbuffers/compat.py` 報錯位置

如果你無法更新套件，或者更新後依然報錯，我們可以直接動手修改那個出錯的函式。

1. 使用 `nano` 編輯器打開這個檔案：

```
bash
```

```
nano /home/user/RSP_demo/demo/env/lib/python3.13/site-packages/flatbuffers/compat.py
```

2. 尋找 `import_numpy()` 這個函式（大概在第 50-60 行左右），你會看到類似這樣的程式碼：

```
python
```

```
def import_numpy():  
    # ... 前面的程式碼 ...  
    try:  
        imp.find_module('numpy')  
        import numpy as np  
        return np  
    except ImportError:  
        return None
```

3. 將它直接簡化修改為只用標準的 `import` 語法，完全繞過 `imp`：

```
python
```

```
def import_numpy():  
    try:  
        import numpy as np  
        return np  
    except ImportError:  
        return None
```

4. 存檔並離開（`Ctrl + O` -> `Enter` -> `Ctrl + X`），接著重新執行程式。

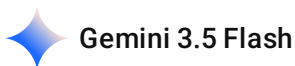
樹莓派作業的終極建議

如果後續 TensorFlow 還是在 Python 3.13 噴出其他大大小小的相容性錯誤（因為 TensorFlow 官方對新版 Python 的支援通常需要一段陣痛期），建議你可以直接**重新建立一個 Python 3.11 或 3.10 的虛擬環境**。在樹莓派上開發深度學習，使用主流、穩定的 Python 版本會讓你的作業寫起來輕鬆非常多！

```
File "/home/user/RSP_demo/train/train_cnn.py", line 13, in
<module>
```

```
~~~~~^
    directory,
    ^^^^^^^^^^^
...<6 lines>...
    verbose=verbose,
    ^^^^^^^^^^^^^^^^^
)
^
```

```
FileNotFoundError: [Errno 2] No such file or directory:
'RSP_demo/dataset'
```



太棒了，前面的 TensorFlow 載入問題已經順利解決了！現在碰到的這個錯誤非常單純：

FileNotFoundError: [Errno 2] No such file or directory: 'RSP_demo/dataset'

這代表 Python 在你目前執行指令的資料夾位置下，找不到 RSP_demo/dataset 這個路徑。

由於你的檔案結構從 Traceback 來看是這樣： `/home/user/RSP_demo/train/train_cnn.py`

當你在 `RSP_demo/train/` 資料夾內執行程式，而程式碼裡寫的是相對路徑 `"RSP_demo/dataset"` 時，Python 就會去尋找 `/home/user/RSP_demo/train/RSP_demo/dataset`，結果自然就找不到。

解決方法

請將 `train_cnn.py`（以及 `train_mobilenet.py`）中的 `DATASET_PATH` 改為 **絕對路徑**，這樣不論你在哪個資料夾執行程式，都不會因為相對位置出錯。

請打開 `train_cnn.py`，將第 8 行左右的：

```
python

DATASET_PATH = "RSP_demo/dataset"
```

修改為絕對路徑：

```
python

DATASET_PATH = "/home/user/RSP_demo/dataset"
```

同理，`train_mobilenet.py` 的路徑也請一併修正為 `/home/user/RSP_demo/dataset`。修改完成後再重新執行，模型應該就能順利讀取資料並開始訓練了！

2026-06-02 11:37:56 You Asked 

how to push all this repository files to another github I created

Gemini 3.5 Flash

要將你目前樹莓派上的整個專案資料夾（包含剛寫好的程式碼、虛擬環境等）推送到一個新的 GitHub 儲存庫（Repository），請在樹莓派的終端機依照以下步驟操作：

步驟 1：先在 `.gitignore` 排除不需要上傳的檔案

在推送之前，**強烈建議**不要把整個虛擬環境（`env/` 資料夾）上傳到 GitHub，因為裡面有幾萬個樹莓派專用的編譯檔案，這會讓上傳變得很慢且沒有意義（別人只需要 `requirements.txt` 即可）。

1. 在 `/home/user/RSP_demo/` 底下建立一個 `.gitignore` 檔案：

```
bash

nano /home/user/RSP_demo/.gitignore
```

2. 將以下內容貼進去，把虛擬環境與訓練好的模型排除：

```
text

env/
*.h5
__pycache__/.DS_Store
```

(註：如果你希望把寫好的 `.h5` 模型也備份到 GitHub，可以把 `*.h5` 那一行刪除。但如果模型檔案太大超過 100MB，GitHub 會拒絕上傳。)

3. 存檔並離開（`Ctrl + O` -> `Enter` -> `Ctrl + X`）。

步驟 2：初始化 Git 並更換遠端倉庫網址

請確保你的終端機目前位在專案的根目錄 `/home/user/RSP_demo`：

```
bash

cd /home/user/RSP_demo
```

接著依序執行以下指令：

1. 初始化 Git（如果以前沒建立過）

```
bash

git init
```

2. 將所有檔案加入暫存區並提交

```
bash

git add .
git commit -m "Initial commit for RSP hand gesture project"
```

3. 設定預設分支名稱為 `main`

```
bash

git branch -M main
```

4. 連結到你的新 GitHub 儲存庫

請先去你新創的 GitHub 頁面，複製你的 Repository 網址（例如：
`https://github.com/你的帳號/你的新倉庫名.git`）。

- 如果你這台樹莓派「從來沒有連結過」舊的 GitHub：

```
bash
```

```
git remote add origin https://github.com/你的帳號/你的新倉庫名.git
```

- 如果你這台樹莓派「之前有連過」別的 GitHub，想要改成新的：

```
bash
```

```
git remote set-url origin https://github.com/你的帳號/你的新倉庫名.git
```

步驟 3：推送到新 GitHub

執行以下指令將程式碼推上雲端：

```
bash
```

```
git push -u origin main
```

💡 **注意事項（關於密碼）：** 當 GitHub 要求你輸入密碼時，請勿輸入你的 GitHub 登入密碼（GitHub 幾年前已禁用密碼推送）。你必須使用 **Personal Access Token (PAT)**：

1. 到 GitHub 網頁 -> **Settings** -> **Developer settings** -> **Personal access tokens** -> **Tokens (classic)**。
2. 產生一個新 Token（勾選 **repo** 權限），複製那串長長的字串。
3. 當終端機跳出 **Password for 'https://github.com...':** 時，把那串 **Token 貼進去** 當作密碼即可成功推送！